

SIS2: Convolutional Neural Networks

Laboratory Report: Dogs vs. Cats Binary Classification

Student 1: Balym Sekenova ID: 23B031433
Student 2: Nursultan Zhantuar ID: 23B030356
Student 3: Maxim Abakarov ID: 23B031182
Student 4: Balzhan Batyrbaeva ID: 23B031245

March 9, 2026

Contents

1	Objective	2
2	Theoretical Background	2
2.1	What is a Convolutional Neural Network?	2
2.2	Key Layers of a CNN	2
2.3	Binary Classification Output	3
2.4	Data Augmentation	3
3	Tools and Dataset	3
4	Part 1. Baseline CNN	4
4.1	Architecture	4
4.2	Training Results	4
4.3	Training Curves	4
5	Part 2. Experiments	6
5.1	Experiment A. Fewer Filters (Simpler Model)	6
5.2	Experiment B. No Pooling Layers (Strided Convolutions)	7
5.3	Experiment C. No Dropout (Overfitting Test)	8
5.4	Experiment D. Different Optimiser (SGD, lr = 0.01)	9
5.5	Experiment E. Deeper Network (4th Conv Block)	10
6	Results Summary	11
7	Answers to Report Questions	12
8	Conclusion	13
A	Full Source Code	15

1 Objective

The objective of this laboratory work is to:

- Understand the architecture and key components of Convolutional Neural Networks (CNNs).
- Build a CNN model for binary image classification (dogs vs. cats) using PyTorch.
- Train the model and evaluate its performance on a validation set.
- Conduct experiments by modifying the network architecture and hyper-parameters, observe how each change affects accuracy and training behaviour, and draw conclusions.

2 Theoretical Background

2.1 What is a Convolutional Neural Network?

A Convolutional Neural Network (CNN) is a class of deep neural networks most commonly applied to image analysis tasks. Unlike regular fully-connected networks, CNNs exploit the spatial structure of images through three key ideas: *local receptive fields*, *shared weights*, and *spatial pooling*. By sliding small learnable filters across the input image, CNNs detect low-level features (edges, textures) in early layers and increasingly abstract features (shapes, object parts) in deeper layers.

2.2 Key Layers of a CNN

- **Convolutional layer (Conv2d):** Applies a set of learnable filters (kernels) to the input. Each filter slides across the image and computes element-wise multiplication followed by summation, producing a feature map. Key parameters: number of filters, kernel size, stride, padding.
- **Activation function (ReLU):** $\text{ReLU}(x) = \max(0, x)$. Introduces non-linearity so the network can learn complex patterns.
- **Pooling layer (MaxPool2d):** Reduces the spatial dimensions of feature maps by taking the maximum value in each local region (e.g. 2×2). This makes the representation more compact and invariant to small translations.
- **Flatten layer:** Converts the 2-D feature maps into a 1-D vector so it can be fed into fully-connected layers.
- **Dense (Fully-Connected) layer:** Standard neural network layer where every neuron is connected to every neuron in the previous layer. Used at the end of the network for classification.
- **Dropout layer:** Randomly sets a fraction of input units to zero during training to prevent overfitting. In this work, a dropout probability of 0.5 is used.

2.3 Binary Classification Output

For a two-class problem (dog or cat), the final layer has a single neuron with no explicit sigmoid activation. Raw logits are produced and the `BCEWithLogitsLoss` criterion (which internally applies sigmoid) is used for numerical stability. During inference, predictions are obtained via $\hat{y} = \sigma(\text{logit}) > 0.5$.

2.4 Data Augmentation

To reduce overfitting on the limited dataset, random transformations are applied to training images on-the-fly: random rotation (up to 20°), random horizontal flip, and random resized crop (scale 0.9–1.0). These operations artificially increase the effective diversity of the training set. Validation images are only resized and normalised, with no augmentation applied.

3 Tools and Dataset

Software: Python 3.10, PyTorch 2.x, torchvision, Matplotlib, NumPy, KaggleHub.

Dataset: A stratified subset of the Kaggle *Dogs vs. Cats* dataset, sourced via `kagglehub`. The subset used was:

- **Training set:** 1 000 images per class (2 000 total).
- **Validation set:** 400 images per class (800 total).

Images were resized to 150×150 pixels and normalised to $[-1, 1]$ using mean and standard deviation of 0.5 per channel. Training used a batch size of 32 for all experiments.

4 Part 1. Baseline CNN

4.1 Architecture

The baseline model consists of three convolutional blocks followed by a fully-connected classifier head. Each convolutional block contains a `Conv2d` layer, a ReLU activation, and a 2×2 `MaxPool2d` layer. The filter progression is $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$ channels. The classifier head flattens the feature maps and applies a `Linear(128 $\times$ 17 $\times$ 17, 128)` layer with ReLU, a 50% Dropout layer, and a final `Linear(128, 1)` output. The Adam optimiser with default learning rate (10^{-3}) was used.

4.2 Training Results

The baseline model was trained for 15 epochs. Selected epoch results are shown below:

Table 1: Baseline model training log (selected epochs)

Epoch	Train Accuracy	Val Accuracy
1	0.5075	0.5962
3	0.6160	0.6462
5	0.6705	0.6325
8	0.7120	0.6725
10	0.7395	0.6937
13	0.7410	0.6713
15	0.7800	0.7288

Final results: Training time: $\approx$ **205 seconds**. Training accuracy: **78.00%** Validation accuracy: **72.88%**

4.3 Training Curves

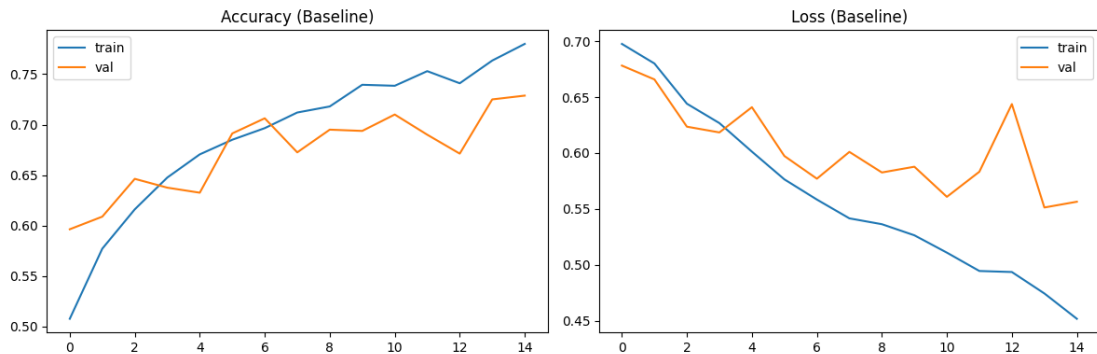


Figure 1: Accuracy and loss curves for the Baseline model over 15 epochs.

The baseline model demonstrates steady learning. The validation accuracy peaks at around 73%, and the gap between training and validation curves remains moderate, which means there is balance between fitting the training data and generalizing to new

data. The loss curves decrease smoothly, and the presence of dropout helps prevent significant overfitting.

5 Part 2. Experiments

All experiments used the same dataset, 15 epochs, batch size 32 and Adam optimiser (unless stated otherwise). Each experiment modifies exactly one aspect of the baseline to isolate its effect.

5.1 Experiment A. Fewer Filters (Simpler Model)

Change: Filter counts reduced from (32, 64, 128) to (8, 16, 32). All other settings are the same as the baseline.

Motivation: Reducing filter counts significantly reduces the number of trainable parameters, which creates a lower-capacity model. This tests whether the baseline has too many parameters for this dataset.

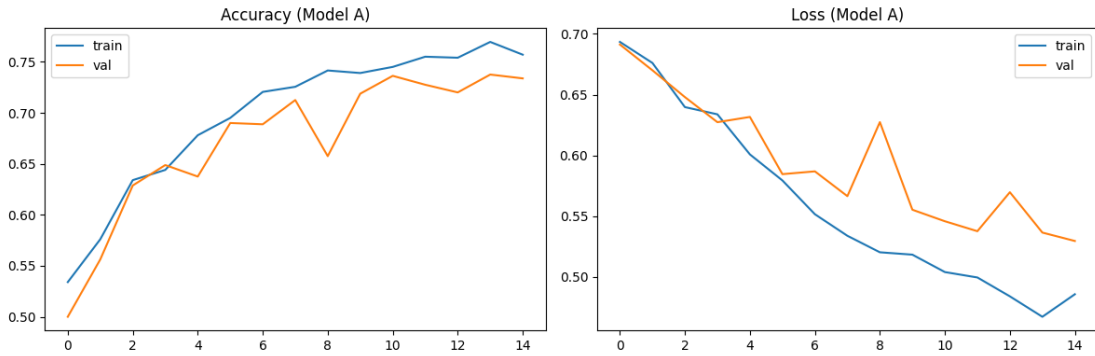


Figure 2: Accuracy and loss curves for Experiment A (fewer filters).

Training process

Epoch 1/15		Train Acc: 0.5340		Val Acc: 0.5000
Epoch 2/15		Train Acc: 0.5760		Val Acc: 0.5563
Epoch 3/15		Train Acc: 0.6340		Val Acc: 0.6288
Epoch 4/15		Train Acc: 0.6440		Val Acc: 0.6488
Epoch 5/15		Train Acc: 0.6780		Val Acc: 0.6375
Epoch 6/15		Train Acc: 0.6950		Val Acc: 0.6900
Epoch 7/15		Train Acc: 0.7205		Val Acc: 0.6887
Epoch 8/15		Train Acc: 0.7255		Val Acc: 0.7125
Epoch 9/15		Train Acc: 0.7415		Val Acc: 0.6575
Epoch 10/15		Train Acc: 0.7390		Val Acc: 0.7188
Epoch 11/15		Train Acc: 0.7450		Val Acc: 0.7362
Epoch 12/15		Train Acc: 0.7550		Val Acc: 0.7275
Epoch 13/15		Train Acc: 0.7540		Val Acc: 0.7200
Epoch 14/15		Train Acc: 0.7695		Val Acc: 0.7375
Epoch 15/15		Train Acc: 0.7570		Val Acc: 0.7338
Training time: 167.70062828063965 seconds				

Results: Final train accuracy: **75.70%**, final val accuracy: **73.38%**. Training time: $\approx$ 168 seconds (faster due to fewer parameters).

Observation: Reducing the number of filters results in a model with lower capacity. Both training and validation accuracy curves rise more slowly and plateau at slightly lower values compared to the baseline. The smaller gap between the two curves suggests less overfitting, but the model also underfits the data, as it cannot capture many complex patterns. Limited representational ability of the model can be seen on the loss curves that decrease but remain higher than the baseline.

5.2 Experiment B. No Pooling Layers (Strided Convolutions)

Change: All MaxPool2d layers removed. Instead, `stride=2` is applied directly in each Conv2d layer to achieve spatial downsampling.

Motivation: Traditional max pooling discards spatial information by taking only the maximum activation. Strided convolutions allow the network to learn how to downsample, which helps potentially preserve more useful information.

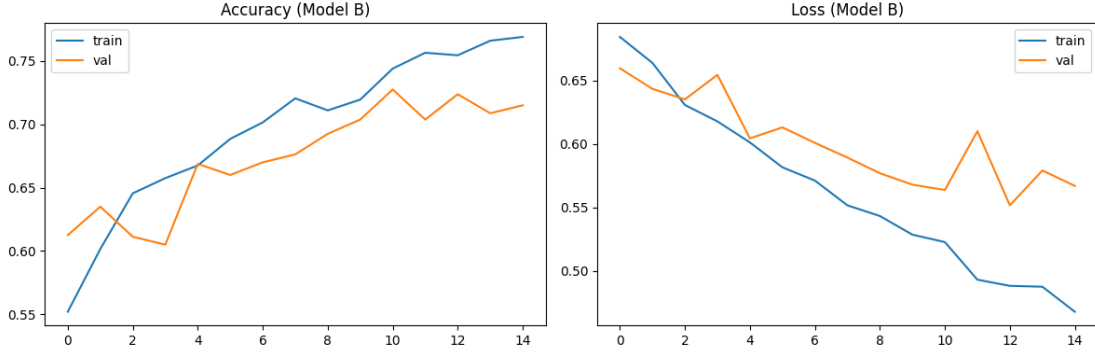


Figure 3: Accuracy and loss curves for Experiment B (strided convolutions, no pooling).

Training process

Epoch 1/15		Train Acc: 0.5520		Val Acc: 0.6125
Epoch 2/15		Train Acc: 0.6015		Val Acc: 0.6350
Epoch 3/15		Train Acc: 0.6455		Val Acc: 0.6112
Epoch 4/15		Train Acc: 0.6575		Val Acc: 0.6050
Epoch 5/15		Train Acc: 0.6675		Val Acc: 0.6687
Epoch 6/15		Train Acc: 0.6885		Val Acc: 0.6600
Epoch 7/15		Train Acc: 0.7015		Val Acc: 0.6700
Epoch 8/15		Train Acc: 0.7205		Val Acc: 0.6763
Epoch 9/15		Train Acc: 0.7110		Val Acc: 0.6925
Epoch 10/15		Train Acc: 0.7195		Val Acc: 0.7037
Epoch 11/15		Train Acc: 0.7440		Val Acc: 0.7275
Epoch 12/15		Train Acc: 0.7565		Val Acc: 0.7037
Epoch 13/15		Train Acc: 0.7545		Val Acc: 0.7238
Epoch 14/15		Train Acc: 0.7660		Val Acc: 0.7087
Epoch 15/15		Train Acc: 0.7690		Val Acc: 0.7150
Training time: 169.28773069381714 seconds				

Results: Final train accuracy: **76.90%**, final val accuracy: **71.50%**. Training time: ≈ 169 seconds.

Observation: Replacing max pooling with strided convolutions produces similar but slightly lower performance compared to the baseline. The accuracy curves are less smooth, and the validation accuracy fluctuates, showing less stable training. The absence of pooling may reduce the model's ability to learn translation-invariant features, and the loss curves reflect this instability by occasional spikes.

5.3 Experiment C. No Dropout (Overfitting Test)

Change: The Dropout(0.5) layer is removed from the classifier head. All other settings identical to the baseline.

Motivation: Dropout is a regularisation technique. Removing it should allow the model to memorise the training set more aggressively. This could lead to severe overfitting.

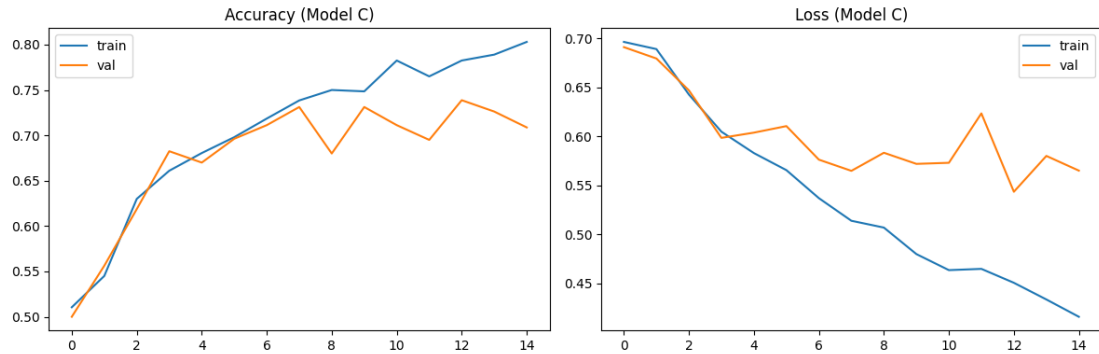


Figure 4: Accuracy and loss curves for Experiment C (no dropout).

Training process

Epoch 1/15		Train Acc: 0.5105		Val Acc: 0.5000
Epoch 2/15		Train Acc: 0.5450		Val Acc: 0.5563
Epoch 3/15		Train Acc: 0.6300		Val Acc: 0.6188
Epoch 4/15		Train Acc: 0.6610		Val Acc: 0.6825
Epoch 5/15		Train Acc: 0.6805		Val Acc: 0.6700
Epoch 6/15		Train Acc: 0.6980		Val Acc: 0.6963
Epoch 7/15		Train Acc: 0.7185		Val Acc: 0.7113
Epoch 8/15		Train Acc: 0.7385		Val Acc: 0.7312
Epoch 9/15		Train Acc: 0.7500		Val Acc: 0.6800
Epoch 10/15		Train Acc: 0.7485		Val Acc: 0.7312
Epoch 11/15		Train Acc: 0.7825		Val Acc: 0.7113
Epoch 12/15		Train Acc: 0.7650		Val Acc: 0.6950
Epoch 13/15		Train Acc: 0.7825		Val Acc: 0.7388
Epoch 14/15		Train Acc: 0.7890		Val Acc: 0.7262
Epoch 15/15		Train Acc: 0.8030		Val Acc: 0.7087
Training time: 186.4713170528412 seconds				

Results: Final train accuracy: **80.30%**, final val accuracy: 70.87%. Training time: $\approx$ 186 seconds.

Observation: Without dropout, the model quickly achieves high training accuracy, but the validation accuracy plateaus and becomes more unstable. The increasing gap between training and validation accuracy is a clear sign of overfitting: the model memorizes the training data but fails to generalize. The loss curves further prove this as validation loss flattens or even increases while training loss continues to decrease.

5.4 Experiment D. Different Optimiser (SGD, lr = 0.01)

Change: Baseline architecture kept intact, but the Adam optimiser is replaced with SGD at a fixed learning rate of 0.01.

Motivation: Adam adapts the learning rate per parameter using moment estimates, whereas SGD uses a fixed global learning rate. This experiment highlights how the choice of the optimiser affects convergence speed and final performance.

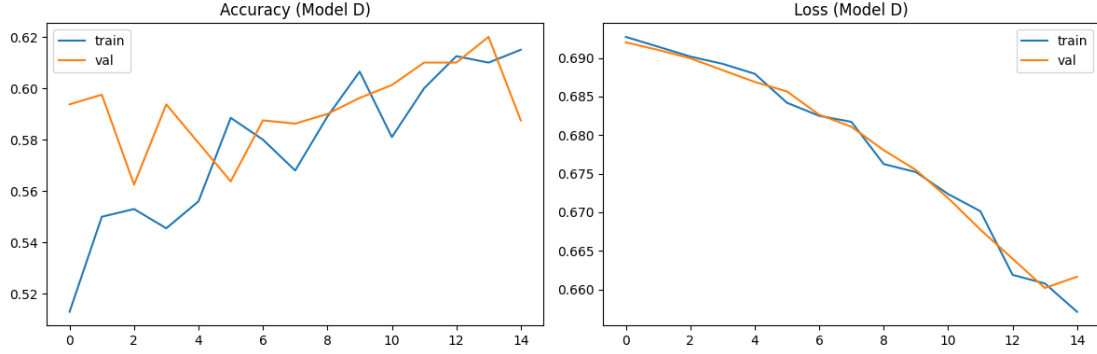


Figure 5: Accuracy and loss curves for Experiment C (no dropout).

Training process

Epoch 1/15		Train Acc: 0.5130		Val Acc: 0.5938
Epoch 2/15		Train Acc: 0.5500		Val Acc: 0.5975
Epoch 3/15		Train Acc: 0.5530		Val Acc: 0.5625
Epoch 4/15		Train Acc: 0.5455		Val Acc: 0.5938
Epoch 5/15		Train Acc: 0.5560		Val Acc: 0.5787
Epoch 6/15		Train Acc: 0.5885		Val Acc: 0.5637
Epoch 7/15		Train Acc: 0.5800		Val Acc: 0.5875
Epoch 8/15		Train Acc: 0.5680		Val Acc: 0.5863
Epoch 9/15		Train Acc: 0.5890		Val Acc: 0.5900
Epoch 10/15		Train Acc: 0.6065		Val Acc: 0.5962
Epoch 11/15		Train Acc: 0.5810		Val Acc: 0.6012
Epoch 12/15		Train Acc: 0.6000		Val Acc: 0.6100
Epoch 13/15		Train Acc: 0.6125		Val Acc: 0.6100
Epoch 14/15		Train Acc: 0.6100		Val Acc: 0.6200
Epoch 15/15		Train Acc: 0.6150		Val Acc: 0.5875
Training time: 184.80421924591064 seconds				

Results: Final train accuracy: **61.50%**, final val accuracy: **58.75%**. Training time: ≈ 184 seconds.

Observation: Switching to SGD with a fixed learning rate results in slower learning. Both training and validation accuracy remain low, barely improving over epochs. The loss curves decrease only slightly, and the model fails to escape the initial plateau. This highlights the importance of optimizer choice, as Adam's adaptive learning rates are much more effective for this task and architecture.

5.5 Experiment E. Deeper Network (4th Conv Block)

Change: A fourth convolutional block with 256 filters is added after the third block, followed by MaxPool2d. The dense layer is widened to `Linear(256×7×7, 256)`.

Motivation: Deeper networks can learn more abstract hierarchical features. This experiment tests whether additional depth improves classification on the dataset.

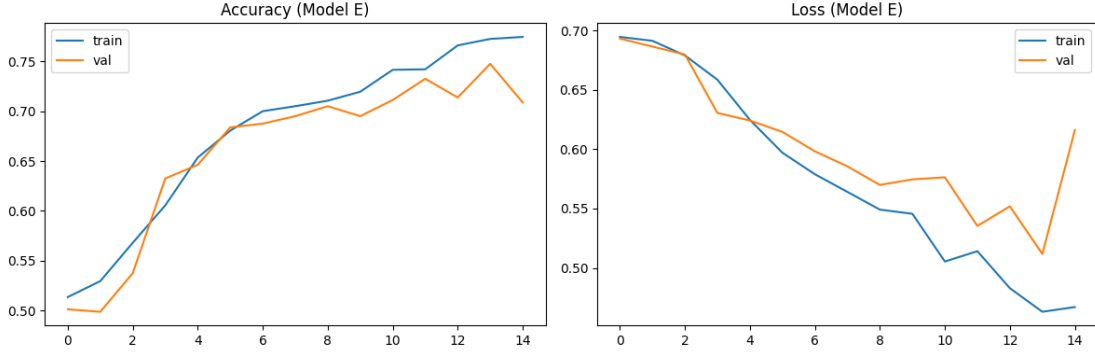


Figure 6: Accuracy and loss curves for Experiment C (no dropout).

Training process

Epoch 1/15		Train Acc: 0.5135		Val Acc: 0.5012
Epoch 2/15		Train Acc: 0.5295		Val Acc: 0.4988
Epoch 3/15		Train Acc: 0.5680		Val Acc: 0.5375
Epoch 4/15		Train Acc: 0.6055		Val Acc: 0.6325
Epoch 5/15		Train Acc: 0.6535		Val Acc: 0.6462
Epoch 6/15		Train Acc: 0.6805		Val Acc: 0.6837
Epoch 7/15		Train Acc: 0.7000		Val Acc: 0.6875
Epoch 8/15		Train Acc: 0.7050		Val Acc: 0.6950
Epoch 9/15		Train Acc: 0.7105		Val Acc: 0.7050
Epoch 10/15		Train Acc: 0.7195		Val Acc: 0.6950
Epoch 11/15		Train Acc: 0.7415		Val Acc: 0.7113
Epoch 12/15		Train Acc: 0.7420		Val Acc: 0.7325
Epoch 13/15		Train Acc: 0.7660		Val Acc: 0.7137
Epoch 14/15		Train Acc: 0.7725		Val Acc: 0.7475
Epoch 15/15		Train Acc: 0.7745		Val Acc: 0.7087
Training time: 188.89886689186096 seconds				

Results: Final train accuracy: 77.45%, final val accuracy: 70.87%. Training time: ≈ 189 seconds.

Observation: Adding a fourth convolutional block increases the model's capacity, but does not improve validation accuracy. The training accuracy rises, but the validation curve is more stagnant and does not surpass the baseline. The loss curves also show more fluctuation. Surprisingly, the validation loss increases abruptly at the last epochs. This suggests that the deeper model overfits the small dataset, and the extra capacity is not used effectively without more data or additional regularization.

6 Results Summary

Table 2: Summary of all experiments

Model	Change	Train Acc.	Val Acc.	Time (s)
Baseline	— (Adam, dropout=0.5, filters 32/64/128)	78.00%	72.88%	205
Model A	Fewer filters (8/16/32)	75.70%	73.38%	168
Model B	No pooling (stride=2)	76.90%	71.50%	169
Model C	No dropout	80.30%	70.87%	186
Model D	SGD, lr=0.01	61.50%	58.75%	184
Model E	Deeper (4 conv blocks, 256 filters)	77.45%	70.87%	189

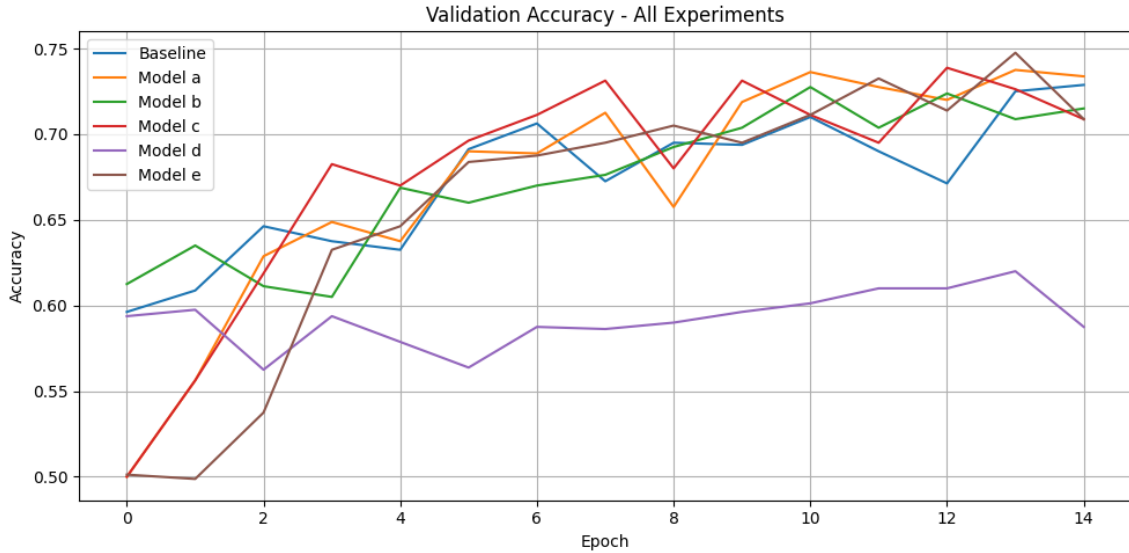


Figure 7: Validation accuracy across all experiments over 15 epochs.

The comparative validation accuracy plot shows that the baseline model, along with Models A, B, C, and E, all achieve similar peak validation accuracies around 0.72–0.75, though Models C and E display more volatility, indicating less stable generalization. Model D, which uses SGD, consistently falls behind, improving only slightly beyond random guessing. Overall, the plot confirms that the baseline’s combination of Adam optimizer and dropout produces the most reliable and consistent performance, while changes in architecture or optimizer can lead to underfitting, overfitting or unstable results.

7 Answers to Report Questions

1. Which experiment gave the highest validation accuracy?

Model A, which used fewer filters (8/16/32), achieved the highest final validation accuracy at **73.38%**, slightly higher than the baseline model's **72.88%**. While the baseline model was more stable across epochs, Model A's validation accuracy curve reached a higher peak, suggesting that a simpler model can sometimes generalize better on small datasets by reducing the risk of overfitting. However, the difference is marginal, and the baseline remains preferable for its consistency and robustness.

2. In Experiment C (no dropout), how do the curves differ? Explain overfitting.

In Experiment C, where dropout was removed, the training accuracy increased rapidly and reached **80.30%** by epoch 15, but the validation accuracy plateaued and became more erratic, ending at **70.87%**. The gap between training and validation accuracy widened as training progressed, and the validation loss curve showed instability while the training loss continued to decrease. For example, after epoch 10, training accuracy was above 75% while validation accuracy fluctuated between 70% and 74%. This pattern is a classic sign of overfitting: the model learns to memorize the training data, including noise and specific details, but fails to extract features that generalize to new, unseen data. Dropout in the baseline model helps prevent this by randomly deactivating neurons during training, forcing the network to learn more robust and distributed representations.

3. How did reducing filters (Experiment A) affect capacity?

Reducing the number of filters in each convolutional layer significantly decreased the model's capacity, resulting in fewer trainable parameters and a simpler network. This led to a final training accuracy of **75.70%** and a validation accuracy of **73.38%**, compared to the baseline's **78.00%** train and **72.88%** validation accuracy. The smaller gap between training and validation accuracy (about 2.3% in Model A vs. 5.1% in the baseline) suggests that the model was less able to memorize the training data, but also less able to capture complex patterns. This experiment highlights the importance of matching model capacity to dataset size: too few parameters can limit learning, while too many can lead to overfitting.

4. Compare Adam vs. SGD (Experiment D). Which is better?

Adam proved to be much more effective than SGD with a fixed learning rate of 0.01. With Adam, the baseline model converged quickly and achieved a validation accuracy of **72.88%** after 15 epochs, while with SGD, both training and validation accuracy remained low, ending at **61.50%** and **58.75%** respectively. The loss curves for SGD showed minimal decrease, indicating slow or stalled learning. Adam's adaptive learning rates allow for more efficient optimization, especially in deeper networks or when the dataset is small and diverse. SGD may require more careful tuning, such as learning rate schedules or momentum, to achieve comparable results.

5. Did the deeper network (Experiment E) improve results?

Adding a fourth convolutional block to create a deeper network did not improve validation accuracy. The deeper model achieved a final training accuracy of **77.45%** and a validation accuracy of **70.87%**, which is lower than the baseline. The in-

creased capacity led to overfitting, as the model was able to fit the training data more closely but failed to generalize to the validation set. The validation accuracy curve was also more volatile, especially in later epochs. This result is consistent with the idea that deeper networks require larger datasets to be effective, and that simply increasing depth without sufficient data or regularization can harm generalization.

6. What is the role of data augmentation?

Data augmentation plays a crucial role in improving model generalization by artificially increasing the diversity of the training data. By applying random transformations such as rotation (up to 20°), flipping, and cropping, the model is exposed to a wider variety of input patterns, making it less likely to memorize specific training examples. This helps reduce overfitting and leads to better performance on unseen data. Without augmentation, the model would likely overfit quickly, as seen in experiments where regularization was reduced or removed.

7. Suggested improvement

A promising improvement would be to add batch normalization layers after each convolutional layer. Batch normalization stabilizes the learning process by normalizing the activations within each mini-batch, allowing for higher learning rates and acting as an additional regularizer. This can lead to faster convergence, smoother loss curves, and potentially higher validation accuracy, especially on small datasets. Other improvements could include learning rate scheduling or using transfer learning with pre-trained models.

8 Conclusion

In the lab, we studied how different approaches to network design and training affect the performance of a convolutional neural network in recognizing two types of images.

We conducted several experiments to evaluate how model size, overfitting prevention methods, the choice of optimization method, and changes to the network design affected the results. It turned out that Model A, which had fewer filters (8/16/32), demonstrated better accuracy on the test data—**73.38%**. This is slightly better than the standard model **72.88%**. The simpler network structure likely reduced overfitting on a small dataset, although the standard model demonstrated more stable performance throughout training.

Using regularization methods is important. In Experiment C, when we removed Dropout, the model began to clearly overfit: accuracy on the training data increased to **80.30%**, but remained at **70.87%** on the test data. This is a large gap, confirming that Dropout helps the network avoid memorizing the training data.

Increasing the number of layers in the model (Experiment E) did not yield good results. The deeper network achieved **77.45%** accuracy on the training data, but only **70.87%** on the validation data. This suggests that the additional capacity led to further overfitting and did not improve the network's generalization ability.

The choice of optimizer also significantly influenced the results. Adam helped the model train quickly and achieve **72.88%** accuracy on the validation data, while SGD with a

constant learning rate of 0.01 performed poorly: only **61.50%** on the training data and **58.75%**, on the validation data.

In summary, our experiments showed that for small datasets, balancing model capacity, proper application of regularization methods, and the use of adaptive optimizers are essential for good network generalization. Future work could explore improving classification results using batch normalization, scheduled learning rate changes, or transfer learning using pre-trained networks.

A Full Source Code

Setup and Data Loading

```
import os
import time
import random
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import kagglehub

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)

# Download dataset
path = kagglehub.dataset_download("tongpython/cat-and-dog")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

IMG_SIZE = 150
BATCH_SIZE = 32
EPOCHS = 15
TRAIN_DIR = "/kaggle/input/cat-and-dog/training_set/training_set"
VAL_DIR = "/kaggle/input/cat-and-dog/test_set/test_set"

train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomRotation(20),
    transforms.RandomHorizontalFlip(),
    transforms.RandomResizedCrop(IMG_SIZE, scale=(0.9, 1.0)),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])

val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])

train_dataset = datasets.ImageFolder(TRAIN_DIR, transform=
    train_transform)
```

```

val_dataset = datasets.ImageFolder(VAL_DIR, transform=
    val_transform)

def stratified_sample(dataset, samples_per_class):
    targets = np.array(dataset.targets)
    indices = []
    for class_idx in np.unique(targets):
        class_indices = np.where(targets == class_idx)[0]
        np.random.shuffle(class_indices)
        indices.extend(class_indices[:samples_per_class])
    return Subset(dataset, indices)

train_sampled = stratified_sample(train_dataset, 1000)
val_sampled = stratified_sample(val_dataset, 400)

train_loader = DataLoader(train_sampled, batch_size=BATCH_SIZE,
    shuffle=True)
val_loader = DataLoader(val_sampled, batch_size=BATCH_SIZE,
    shuffle=False)

```

Training and Plotting Utilities

```
def train_model(model, optimizer):
    model.to(device)
    criterion = nn.BCEWithLogitsLoss()
    train_acc_list, val_acc_list = [], []
    train_loss_list, val_loss_list = [], []
    start_time = time.time()

    for epoch in range(EPOCHS):
        model.train()
        correct, total, running_loss = 0, 0, 0

        for images, labels in train_loader:
            images = images.to(device)
            labels = labels.float().unsqueeze(1).to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            preds = torch.sigmoid(outputs) > 0.5
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        train_acc = correct / total
        train_loss = running_loss / len(train_loader)

        model.eval()
        correct, total, running_loss = 0, 0, 0
        with torch.no_grad():
            for images, labels in val_loader:
                images = images.to(device)
                labels = labels.float().unsqueeze(1).to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)
                running_loss += loss.item()
                preds = torch.sigmoid(outputs) > 0.5
                correct += (preds == labels).sum().item()
                total += labels.size(0)

        val_acc = correct / total
        val_loss = running_loss / len(val_loader)
        train_acc_list.append(train_acc); val_acc_list.append(
            val_acc)
        train_loss_list.append(train_loss); val_loss_list.append(
            val_loss)
        print(f"Epoch_{epoch+1}/{EPOCHS}|_|"
              f"Train_Acc:_{train_acc:.4f}|_|Val_Acc:_{val_acc:.4"
              f}")
```

```

print("Training_time:", time.time() - start_time, "seconds")
return train_acc_list, val_acc_list, train_loss_list,
       val_loss_list

def plot_history(train_acc, val_acc, train_loss, val_loss, title=
    ""):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].plot(train_acc, label='train')
    axes[0].plot(val_acc, label='val')
    axes[0].set_title("Accuracy_" + title); axes[0].legend()
    axes[1].plot(train_loss, label='train')
    axes[1].plot(val_loss, label='val')
    axes[1].set_title("Loss_" + title); axes[1].legend()
    plt.tight_layout(); plt.show()

```

Baseline Model

```
class BaselineCNN(nn.Module):
    def __init__(self):
        super(BaselineCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 17 * 17, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 1)
        )
    def forward(self, x):
        return self.classifier(self.features(x))

model = BaselineCNN()
optimizer = optim.Adam(model.parameters())
train_acc, val_acc, train_loss, val_loss = train_model(model,
    optimizer)
plot_history(train_acc, val_acc, train_loss, val_loss, "(Baseline
)")
```

Experiment A — Fewer Filters

```
class ModelA(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 8, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 17 * 17, 128),
            nn.ReLU(), nn.Dropout(0.5), nn.Linear(128, 1)
        )
    def forward(self, x):
        return self.classifier(self.features(x))

model_A = ModelA()
optimizer_A = optim.Adam(model_A.parameters())
train_acc_a, val_acc_a, train_loss_a, val_loss_a = train_model(
    model_A, optimizer_A)
plot_history(train_acc_a, val_acc_a, train_loss_a, val_loss_a, "(
    Model_A)")
```

Experiment B — No Pooling (Strided Convolutions)

```
class ModelB(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2), nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2), nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2), nn.ReLU()
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 17 * 17, 128),
            nn.ReLU(), nn.Dropout(0.5), nn.Linear(128, 1)
        )
    def forward(self, x):
        return self.classifier(self.features(x))

model_B = ModelB()
optimizer_B = optim.Adam(model_B.parameters())
train_acc_b, val_acc_b, train_loss_b, val_loss_b = train_model(
    model_B, optimizer_B)
plot_history(train_acc_b, val_acc_b, train_loss_b, val_loss_b, "(
    Model_B)")
```

Experiment C — No Dropout

```
class ModelC(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 17 * 17, 128),
            nn.ReLU(),
            nn.Linear(128, 1)    # No Dropout
        )
    def forward(self, x):
        return self.classifier(self.features(x))

model_C = ModelC()
optimizer_C = optim.Adam(model_C.parameters())
train_acc_c, val_acc_c, train_loss_c, val_loss_c = train_model(
    model_C, optimizer_C)
plot_history(train_acc_c, val_acc_c, train_loss_c, val_loss_c, "(
    Model_C)")
```

Experiment D — SGD Optimiser

```
# Same architecture as Baseline, different optimiser
model_D = BaselineCNN()
optimizer_D = optim.SGD(model_D.parameters(), lr=0.01)
train_acc_d, val_acc_d, train_loss_d, val_loss_d = train_model(
    model_D, optimizer_D)
```

Experiment E — Deeper Network

```
class ModelE(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128, 256, 3), nn.ReLU(), nn.MaxPool2d(2) #
            4th block
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256 * 7 * 7, 256),
            nn.ReLU(), nn.Dropout(0.5), nn.Linear(256, 1)
```

```

    )
    def forward(self, x):
        return self.classifier(self.features(x))

model_E = ModelE()
optimizer_E = optim.Adam(model_E.parameters())
train_acc_e, val_acc_e, train_loss_e, val_loss_e = train_model(
    model_E, optimizer_E)
plot_history(train_acc_e, val_acc_e, train_loss_e, val_loss_e, "(
    Model_E)")

```

Comparative Validation Accuracy Plot

```

plt.figure(figsize=(10, 5))
for name, metrics in models_metrics.items():
    val_acc = metrics["val_acc"]
    plt.plot(val_acc, label=" ".join(name.split("_")).capitalize(
        ))
plt.title('Validation Accuracy - All Experiments')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```